

Data Assimilation in Nonlinear Fluid Flow IV

Neural surrogate dynamics: small accuracy penalty for a much cheaper forecast at scale

Presentation 4 of 4

1. Learning objectives

- Formulate a neural surrogate for the one-step nonlinear flow map.
- Train a small local MLP with only NumPy.
- Combine the surrogate with nudging and derive an error bound.
- Separate two questions: **accuracy at a trained resolution** and **computational scaling**.

2. The remaining bottleneck: one PDE forecast can still be expensive

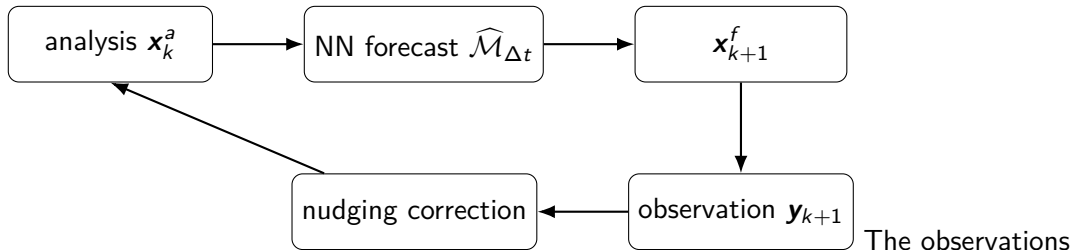
DA design	expensive PDE forecasts/cycle	online forecast object
EnKF, $M = 40$	40	nonlinear solver ensemble
PDE + nudging	1	nonlinear solver
NN + nudging	0	learned map $\widehat{\mathcal{M}}_{\Delta t}$

At high resolution, even the single forecast in nudging can dominate wall time.

Key idea

The goal is not to remove data assimilation; it is to replace the expensive forecast operator while keeping observations in the loop.

3. Hybrid architecture



repeatedly correct both initial-condition error and surrogate-model drift.

4. What exactly is learned?

Instead of learning instantaneous derivatives,

$$\dot{\mathbf{x}} = \mathcal{F}(\mathbf{x}),$$

we learn the direct map over one assimilation interval,

$$\widehat{\mathbf{x}}_{k+1} = \widehat{\mathcal{M}}_{\Delta t}(\mathbf{x}_k).$$

For the tutorial, $\Delta t = 0.10$.

Computational benefit

One NN call can replace many stability-limited PDE substeps inside the same physical interval.

5. A small local neural stencil

At grid point i , collect a 9-point periodic neighborhood,

$$\mathbf{z}_i = [u_{i-4}, \dots, u_i, \dots, u_{i+4}]^\top.$$

A shared MLP predicts the local state increment,

$$\Delta u_i = W_2 \tanh(W_1 \mathbf{z}_i + b_1) + b_2.$$

Then

$$\hat{u}_i^{k+1} = u_i^k + \Delta u_i.$$

The same network is applied at every spatial location.

6. Why shared local weights?

This architecture is intentionally simple:

- parameter count does not grow with the number of grid points,
- periodic translation symmetry is respected,
- inference is vectorizable,
- the model resembles a learned nonlinear numerical stencil,
- implementation needs only matrix multiplication and tanh.

It is not presented as a state-of-the-art neural operator; it is a transparent presentation surrogate.

7. Training data

We generate many smooth random Burgers states and their exact one-interval forecasts:

$$(\mathbf{u}^j, \mathcal{M}_{\Delta t}(\mathbf{u}^j)).$$

To improve robustness to assimilation states, each training state is also perturbed by small Gaussian noise before forecasting.

Key idea

The network sees not only clean trajectories, but also off-trajectory states similar to those produced by imperfect DA estimates.

8. Training objective

The network predicts the increment

$$\Delta \mathbf{u} = \mathcal{M}_{\Delta t}(\mathbf{u}) - \mathbf{u}.$$

We minimize normalized mean-square error,

$$\mathcal{L}(\theta) = \frac{1}{B} \sum_{j=1}^B \left\| \widehat{\Delta \mathbf{u}}_{\theta}^{(j)} - \Delta \mathbf{u}^{(j)} \right\|_2^2.$$

Inputs and targets are standardized using training-set mean and standard deviation.

9. No heavy ML framework is required

The included implementation uses:

- NumPy for arrays and matrix products,
- a hand-written tanh MLP,
- hand-written backpropagation,
- hand-written Adam optimizer.

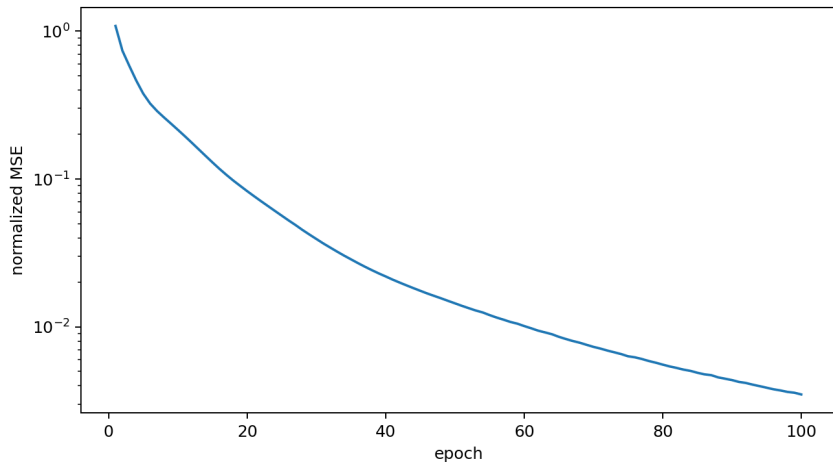
No PyTorch, TensorFlow, JAX, SciPy, or scikit-learn is needed.

10. Core neural forecast code

```
X = local_windows(u) # [N, 9]
Xn = (X - x_mean) / x_std
h = tanh(Xn @ W1 + b1)
delta = (h @ W2 + b2) * y_std + y_mean
u_next = u + delta.ravel()
```

Full training/inference code: `code/surrogate.py`.

11. Training curve



The optimizer reduces one-step normalized error rapidly on this small synthetic dataset.

12. One-step accuracy is not enough

Even a small local error can accumulate under free rollout:

$$\mathbf{e}_{k+1} = \widehat{\mathcal{M}}(\hat{\mathbf{x}}_k) - \mathcal{M}(\mathbf{x}_k).$$

Add and subtract $\widehat{\mathcal{M}}(\mathbf{x}_k)$:

$$\|\mathbf{e}_{k+1}\| \leq L_{\widehat{\mathcal{M}}} \|\mathbf{e}_k\| + \left\| \widehat{\mathcal{M}}(\mathbf{x}_k) - \mathcal{M}(\mathbf{x}_k) \right\|.$$

This is why recurrent correction by observations is valuable.

13. Hybrid DA update

Forecast with the learned map,

$$\mathbf{x}_{k+1}^f = \widehat{\mathcal{M}}_{\Delta t}(\mathbf{x}_k^a),$$

then nudge,

$$\mathbf{x}_{k+1}^a = \mathbf{x}_{k+1}^f + \gamma G(\mathbf{y}_{k+1} - H\mathbf{x}_{k+1}^f).$$

The correction now plays two roles:

- 1 synchronize an uncertain initial state,
- 2 repeatedly remove neural-surrogate drift.

14. Theorem: bounded hybrid error with bounded surrogate error

Theorem

Assume the exact map is L -Lipschitz, the surrogate satisfies

$$\|\widehat{\mathcal{M}}(x) - \mathcal{M}(x)\| \leq \varepsilon,$$

and the analysis correction contracts forecast error by factor $\rho < 1$, with additive observation term bounded by η . Then

$$\|e_{k+1}\| \leq \rho L \|e_k\| + \rho\varepsilon + \eta.$$

If $a = \rho L < 1$, the asymptotic error is bounded by

$$\limsup_k \|e_k\| \leq \frac{\rho\varepsilon + \eta}{1 - a}.$$

15. Proof sketch

Before assimilation,

$$\begin{aligned}\|e_{k+1}^f\| &= \|\widehat{\mathcal{M}}(\hat{x}_k^a) - \mathcal{M}(x_k)\| \\ &\leq \|\widehat{\mathcal{M}}(\hat{x}_k^a) - \widehat{\mathcal{M}}(x_k)\| + \|\widehat{\mathcal{M}}(x_k) - \mathcal{M}(x_k)\| \\ &\leq L \|e_k^a\| + \varepsilon.\end{aligned}$$

After a contractive observation update,

$$\|e_{k+1}^a\| \leq \rho \|e_{k+1}^f\| + \eta.$$

Substitute and iterate the geometric recursion. $\square$

16. Interpretation of the bound

The bound separates three effects:

- L : dynamical amplification of existing state error,
- ε : surrogate approximation error,
- ρ : strength of observation-induced contraction.

Key idea

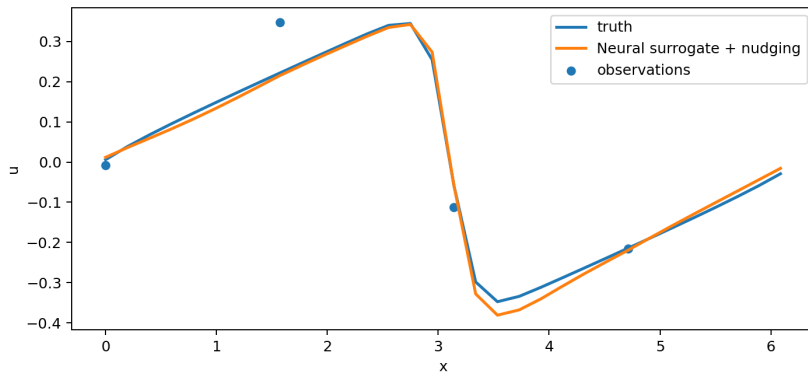
A surrogate does not need perfect long-horizon free-running accuracy if observations arrive often enough to keep $\rho L < 1$.

17. Same Burgers DA experiment

The neural surrogate is trained at the same $N = 32$ resolution used for the accuracy experiment.

Forecast interval	$\Delta t = 0.10$
Network input	9 local values
Hidden width	64
Network output	local Δu
Assimilation	nudging, $\gamma = 0.10$
Sensors	4 of 32 grid points

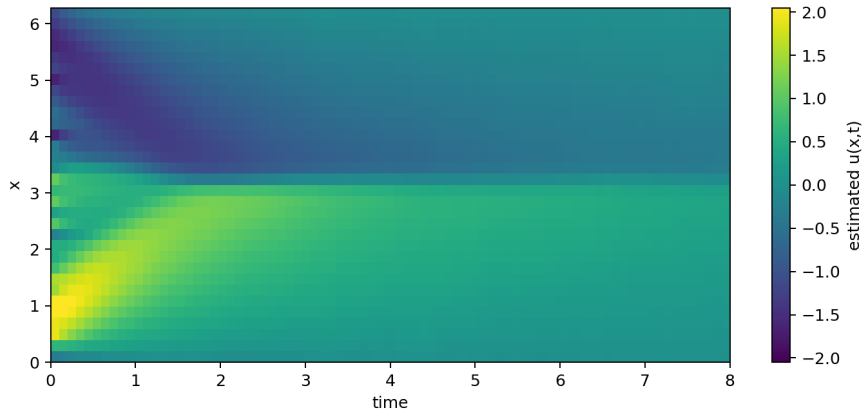
18. Final state: neural surrogate + nudging



Key idea

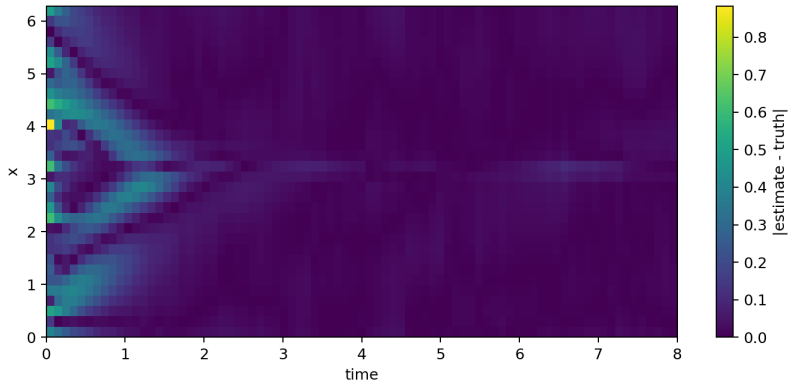
The learned forecast keeps the analysis close to the truth even though every online PDE forecast has been removed from the assimilation loop.

19. NN + nudging output over the complete trajectory



The output map includes a colorbar for the estimated $u(x, t)$ field and shows that the learned dynamics preserve the dominant moving structures.

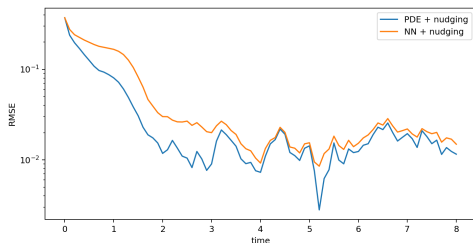
20. NN + nudging absolute error map



Important

The colorbar shows $|u_{\text{NN+DA}} - u_{\text{truth}}|$. The residual is larger than with the exact PDE forecast, but it remains far below the failed linear-KF regime.

21. Accuracy penalty from replacing the PDE forecast



Method	Mean RMSE	Online PDE solves/cycle	Difference
PDE + nudging	0.035	1	baseline
NN + nudging	0.054	0	+0.019 RMSE

Key idea

The toy experiment accepts a modest absolute error increase in exchange for replacing the expensive online PDE forecast by one neural evaluation.

22. Four-method tradeoff summary

Method	RMSE	PDE solves	UQ	Main limitation
Frozen KF	0.647	0*	covariance	nonlinear model mismatch
EnKF $M = 40$	0.012	40	yes	compute and memory
PDE + nudging	0.035	1	no	prescribed gain
NN + nudging	0.054	0	no	surrogate generalization

* Cheap frozen linear forecast; no nonlinear Burgers solve.

Key idea

The design point moves along four axes: accuracy, uncertainty, online compute, and robustness.

23. CFL-limited explicit PDE stepping

For advection, an explicit solver often requires

$$\delta t_{\text{PDE}} \lesssim C \frac{\Delta x}{|u|}.$$

On a fixed physical domain, $\Delta x \propto N^{-1}$, so a fixed physical forecast interval may require

$$N_{\text{sub}} \propto N.$$

If each spatial update costs $\mathcal{O}(N)$ in 1-D, then

$$C_{\text{PDE}}(N) \approx \mathcal{O}(N^2)$$

for this explicit fixed-interval benchmark.

24. Direct neural forecast: different online scaling mechanism

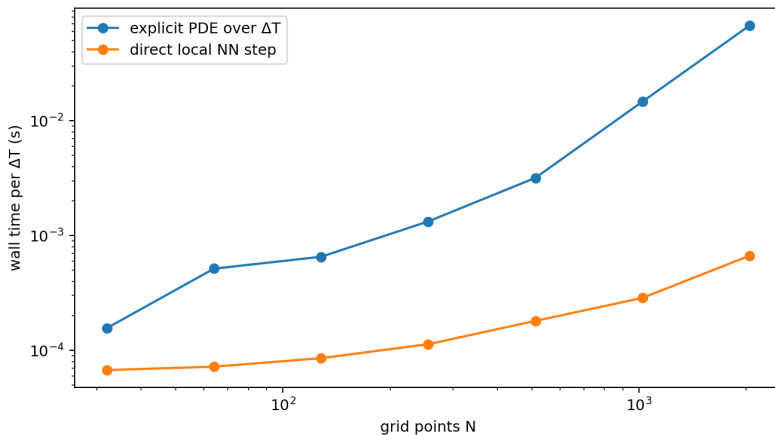
A shared local MLP applies a fixed amount of work per grid point,

$$C_{\text{NN}}(N) = \mathcal{O}(Nh),$$

with hidden width h fixed, and predicts the complete assimilation interval directly.

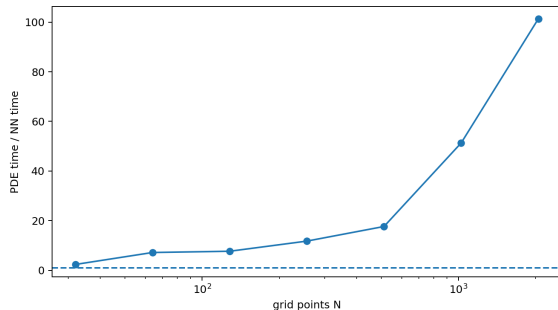
Cost component	PDE forecast	Neural forecast
CFL substeps	grows as grid refines	none
Online operator calls	many RHS evaluations	one NN map
Offline cost	none beyond solver setup	training required
Physics guarantees	inherited from solver	must be tested / enforced

25. Timing benchmark versus domain size



The benchmark compares computation for one fixed physical interval. It measures **runtime scaling**, not cross-resolution predictive accuracy.

26. Measured speedup grows with resolution in this toy benchmark



At $N = 2048$, the generated NumPy benchmark gives approximately **101.4** $\times$ lower direct-step time for the local NN.

Important

The exact ratio is hardware- and implementation-dependent. The robust conclusion is the different scaling mechanism, not the precise number.

27. Neural-surrogate tradeoffs before using it in production

Dimension	Benefit	Risk / validation need
Online speed	very low forecast latency	training cost is paid offline
Large domains	avoids CFL substepping	memory bandwidth can dominate
Accuracy	small penalty in this test	distribution shift can increase error
Physics	learns nonlinear finite-time map	conservation / stability not automatic
DA feedback	observations repeatedly correct drift	sparse sensors may not rescue severe model error
Deployment	simple NumPy inference here	realistic models may need accelerators

28. Presentation-series synthesis

Method	Main strength	Main cost / assumption	Best fit here
KF	exact linear–Gaussian inference	linearity and dense covariance	small, near-linear systems
EnKF	nonlinear, flow-dependent covariance	many model trajectories	accuracy plus uncertainty
Nudging	one-trajectory feedback correction	prescribed gain; no posterior spread	low-cost state estimation
NN + nudging	removes online PDE forecast	training and generalization risk	large-domain, low-latency DA

The engineering question is: which accuracy–uncertainty–compute tradeoff matches the application?